

ХЕРСОНСЬКИЙ НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ ТА ДИЗАЙНУ КАФЕДРА
ПРОГРАМНИХ ЗАСОБІВ І ТЕХНОЛОГІЙ

Лабораторна робота №9
з дисципліни:
«Веб-програмування та веб-дизайн»

студент гр.2ПР1

Р.В.Семеніхін

Перевірів

О.С.Комісаров

Тема: ФОРМИ: ОБРОБКА ТА ВАЛІДАЦІЯ

Мета: формування практичних навичок обробки HTML-форм засобами JavaScript: реалізація клієнтської валідації даних, динамічне відображення повідомлень про помилки в режимі реального часу, управління станом кнопки надсилання під час «запиту на сервер» та серіалізація даних форми через об'єкт FormData.

ХІД РОБОТИ

1. Підготовка середовища та структури проєкту

На початковому етапі було налаштовано робоче оточення для розробки.

Виконано наступні дії:

1. Створено робочу директорію проєкту з назвою lab08-forms.
2. У середовищі VS Code ініціалізовано порожній Git-репозиторій за допомогою команди `git init` у вбудованому терміналі.
3. Створено три базові файли, що складають структуру веб-додатка:
`index.html` — для розмітки структури форми;
`style.css` — для візуального оформлення та стилізації станів валідації;
`app.js` — для написання логіки обробки подій та функцій перевірки.

index.html

```
<link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="form-card">
    <form id="registrationForm" novalidate>
      <h2>Реєстрація</h2>

      <div class="form-group">
        <label for="name">Повне ім'я</label>
        <input type="text" id="name" name="name"
placeholder="Введіть ім'я">
        <span class="error-msg" id="nameError"></span>
      </div>

      <div class="form-group">
        <label for="email">Email</label>
        <input type="email" id="email" name="email"
placeholder="example@mail.com">
        <span class="error-msg" id="emailError"></span>
      </div>

      <div class="form-group">
        <label for="password">Пароль</label>
        <input type="password" id="password"
name="password" placeholder="Мінімум 8 символів">
        <span class="error-msg" id="passwordError"></span>
      </div>
    </form>
  </div>
</body>
```

```

        <div class="form-group">
            <label for="confirm">Підтвердження пароля</label>
            <input type="password" id="confirm" name="confirm"
placeholder="Повторіть пароль">
            <span class="error-msg" id="confirmError"></span>
        </div>

        <div class="form-group">
            <label for="role">Ваша роль</label>
            <select id="role" name="role">
                <option value="">Оберіть роль...</option>
                <option value="student">Студент</option>
                <option value="teacher">Викладач</option>
                <option value="admin">Адміністратор</option>
            </select>
            <span class="error-msg" id="roleError"></span>
        </div>

        <div class="form-group checkbox-group">
            <input type="checkbox" id="terms" name="terms">
            <label for="terms">Я погоджуюсь з умовами</label>
            <span class="error-msg" id="termsError"></span>
        </div>

        <button type="submit"
id="submitBtn">Зареєструватися</button>
    </form>
    <div id="successMessage" style="display: none;">
        <h3>Реєстрація успішна! 🎉</h3>
        <p id="welcomeUser"></p>
    </div>
</div>

<script src="app.js" defer></script>
</body>
</html>

```

style.css

```

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
    background-color: #f0f2f5;
    margin: 0;
}

.form-card {
    background: white;
    padding: 2rem;
    border-radius: 12px;
}

```

```

        box-shadow: 0 4px 15px rgba(0,0,0,0.1);
        width: 100%;
        max-width: 400px;
    }

    .form-group {
        margin-bottom: 1.5rem;
        display: flex;
        flex-direction: column;
    }

    input, select {
        padding: 10px;
        border: 2px solid #ccc;
        border-radius: 6px;
        outline: none;
        transition: border-color 0.3s;
    }

    /* Стани валідації */
    .invalid input, .invalid select { border-color: #e74c3c; }
    .valid input, .valid select { border-color: #27ae60; }

    .error-msg {
        color: #e74c3c;
        font-size: 0.85em;
        margin-top: 5px;
        min-height: 1em;
    }

    button {
        width: 100%;
        padding: 12px;
        background-color: #3498db;
        color: white;
        border: none;
        border-radius: 6px;
        cursor: pointer;
        font-weight: bold;
    }

    button:hover { background-color: #2980b9; }
    button:disabled { opacity: 0.6; cursor: not-allowed; }

```

app.js

```

const EMAIL_REGEX = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

const validators = {
    name: (val) => val.trim().length < 2 ? "Ім'я занадто коротке"
    : null,
    email: (val) => !EMAIL_REGEX.test(val) ? "Невірний формат
email" : null,

```

```

        password: (val) => val.length < 8 ? "Мінімум 8 символів" :
null,
        confirm: (val, pass) => val !== pass ? "Паролі не збігаються"
: null,
        role: (val) => val === "" ? "Оберіть роль" : null,
        terms: (val, checked) => !checked ? "Потрібна згода" : null
    };
function showError(fieldId, message) {
    const group = document.getElementById(fieldId).closest('.form-
group');
    const errorSpan = document.getElementById(`${fieldId}Error`);
    group.classList.remove('valid');
    group.classList.add('invalid');
    errorSpan.textContent = message;
}

function clearError(fieldId) {
    const group = document.getElementById(fieldId).closest('.form-
group');
    const errorSpan = document.getElementById(`${fieldId}Error`);
    group.classList.remove('invalid');
    group.classList.add('valid');
    errorSpan.textContent = "";
}
const form = document.getElementById('registrationForm');
const submitBtn = document.getElementById('submitBtn');

// Функція перевірки всього відразу
function validateAll() {
    let isValid = true;
    const pass = document.getElementById('password').value;

    // Перевірка кожного поля (спрощено для розуміння)
    const fields = ['name', 'email', 'password', 'role'];
    fields.forEach(id => {
        const val = document.getElementById(id).value;
        const err = validators[id](val);
        err ? (showError(id, err), isValid = false) :
clearError(id);
    });

    // Окремо для confirm та terms
    const confirmVal = document.getElementById('confirm').value;
    const confirmErr = validators.confirm(confirmVal, pass);
    confirmErr ? (showError('confirm', confirmErr), isValid =
false) : clearError('confirm');

    const termsChecked = document.getElementById('terms').checked;
    const termsErr = validators.terms(null, termsChecked);
    termsErr ? (showError('terms', termsErr), isValid = false) :
clearError('terms');

    return isValid;
}

```

```

}

// Обробка натискання кнопки
form.addEventListener('submit', async (e) => {
    e.preventDefault();

    if (!validateAll()) return;

    // Імітація завантаження
    submitBtn.disabled = true;
    submitBtn.textContent = "Надсилання...";

    await new Promise(res => setTimeout(res, 2000));

    // Збір даних
    const formData = new FormData(form);
    const data = Object.fromEntries(formData.entries());

    console.log("JSON Дані форми:", JSON.stringify(data, null,
2));

    // Успішне завершення
    form.style.display = "none";
    const successBox = document.getElementById('successMessage');
    successBox.style.display = "block";
    document.getElementById('welcomeUser').textContent = `Вітаємо,
${data.name}!`;
});

// Жива валідація при введенні
document.querySelectorAll('input, select').forEach(el => {
    el.addEventListener('input', () => {
        // Тут можна викликати validateAll() для миттєвого
оновлення
        validateAll();
    });
});
});

```

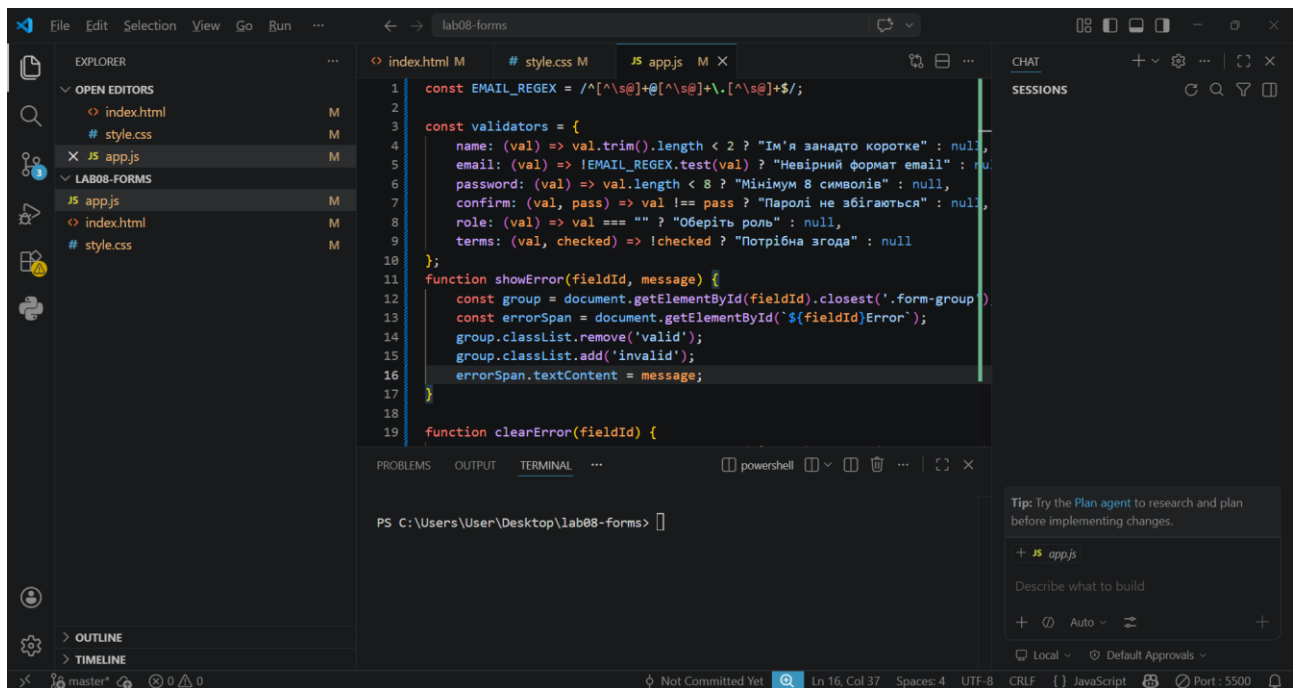


Рис.1 VS Code з трьома відкритими файлами та запущеним портом 5500

Проект запущено за допомогою розширення Live Server на локальному хості (http://127.0.0.1:5500), що дозволило спостерігати зміни в коді у реальному часі без ручного оновлення сторінки.

2. Розмітка HTML-форми та її стилізація

Для створення інтерфейсу користувача було реалізовано HTML-структуру форми з урахуванням сучасних стандартів доступності та зручності:

1. У файлі index.html додано тег <form> з атрибутом novalidate. Це необхідно для того, щоб вимкнути стандартну браузерну валідацію та повністю контролювати процес через JavaScript.
2. Додано шість полів введення з відповідними ідентифікаторами (id) та атрибутами name:
 - Текстове поле для імені;
 - Поле типу email;
 - Поля для пароля та його підтвердження (type="password");
 - Випадаючий список (select) для вибору ролі;
 - Чекбокс для згоди з умовами.
3. Після кожного поля розміщено спеціальний контейнер , який призначений для динамічного виведення текстових повідомлень про помилки.
4. У файлі style.css описано стилі для різних станів:
 - Клас .valid (зелена рамка) — для коректно заповнених полів;
 - Клас .invalid (червона рамка) — для полів з помилками.
 - Стилi для кнопки відправки, включаючи стан :disabled (напівпрозорість), що активується під час імітації надсилання даних.

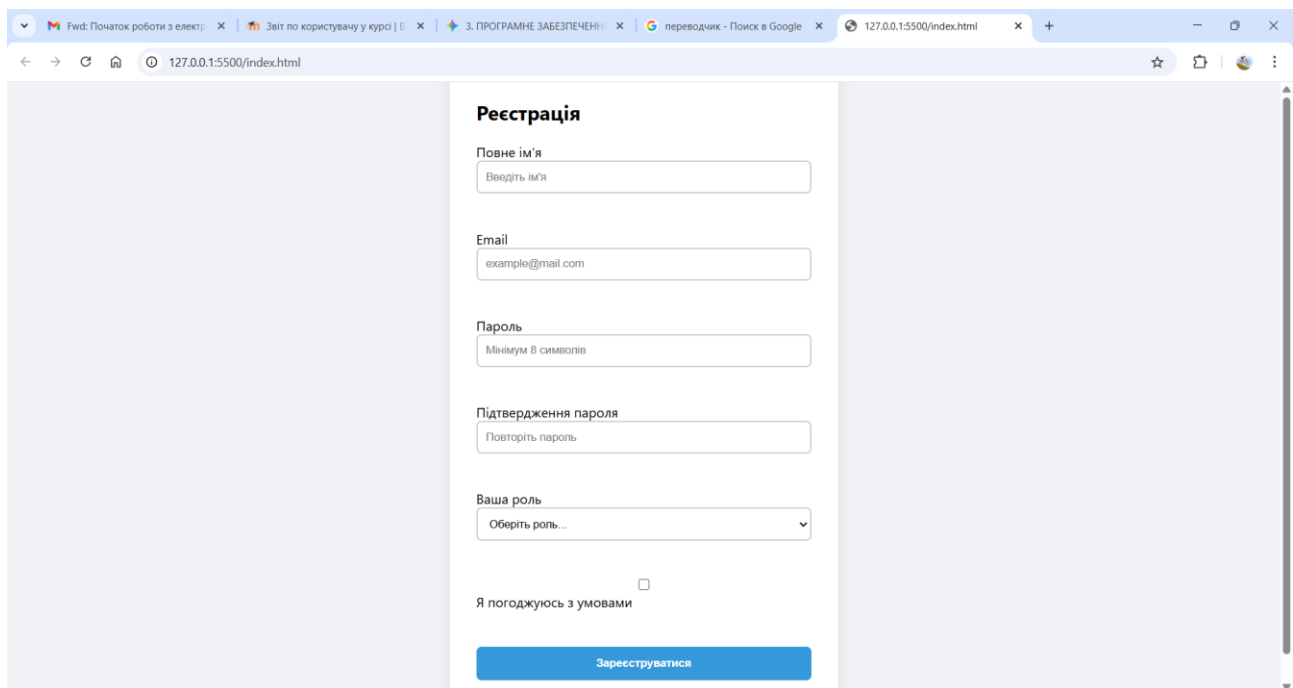


Рис.2 Відображення готової форми в браузері

Тест №1: Валідація при спробі надсилання порожньої форми

Нижче продемонстровано роботу алгоритму валідації в момент натискання кнопки «Зареєструватися» без попереднього заповнення полів.

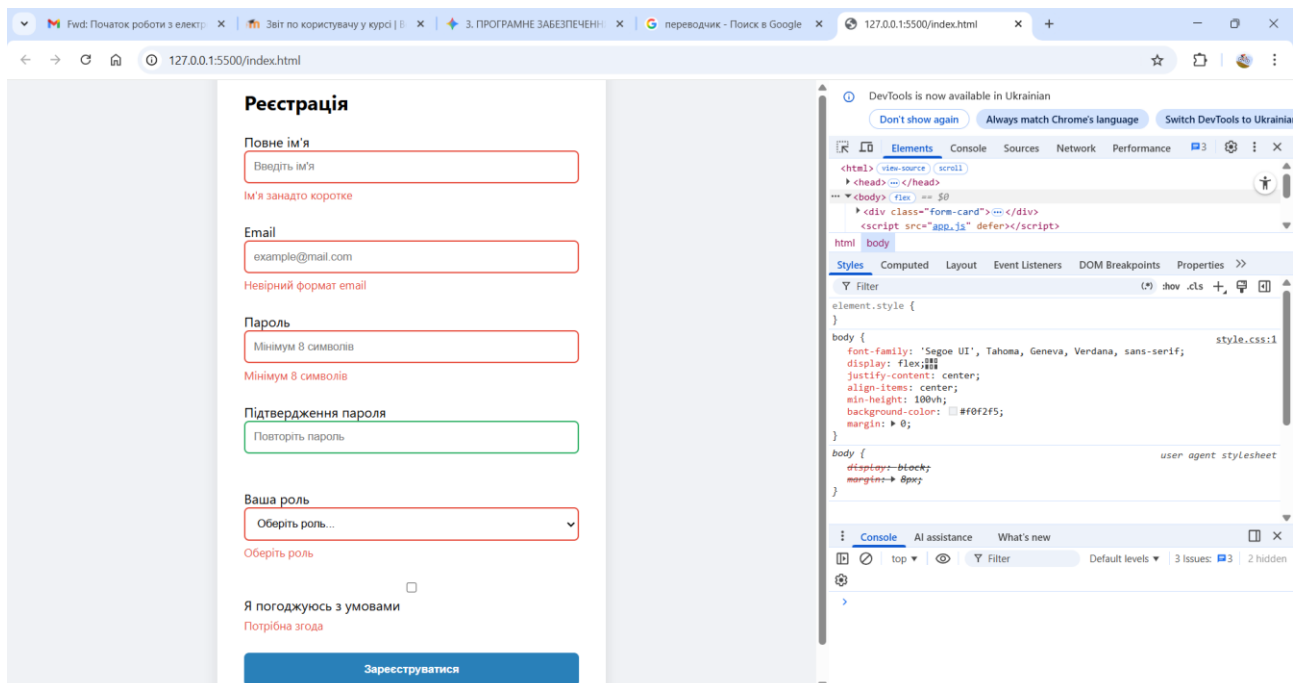


Рис.3 перевірка клієнтської валідації при спробі відправлення порожньої форми.

Опис результату:

Спрацювання функції `validateAll()`: При натисканні на кнопку спрацював обробник події `submit`, який викликав комплексну перевірку всіх полів форми одночасно.

Візуальна індикація помилок:

Поля «Повне ім'я», «Email», «Пароль» та випадаючий список «Ваша роль» змінили колір рамки на червоний (клас `.invalid`).

Під кожним невалідним полем з'явилися відповідні текстові повідомлення: "Ім'я занадто коротке", "Невірний формат email", "Мінімум 8 символів", "Оберіть роль".

Чекбокс отримав попередження "Потрібна згода".

Блокування відправки: Оскільки функція валідації повернула значення `false`, виконання скрипта було зупинено командою `return`, і імітація мережевого запиту не розпочалася. Це підтверджує порожня консоль у правій частині скріншоту.

Стан інструментів розробника: В панелі Elements видно структуру документа, а в секції Styles відображені CSS-властивості для `body`, що забезпечують центрування форми на сторінці.

Тест №2: Валідація збігу паролів та коректності заповнення полів

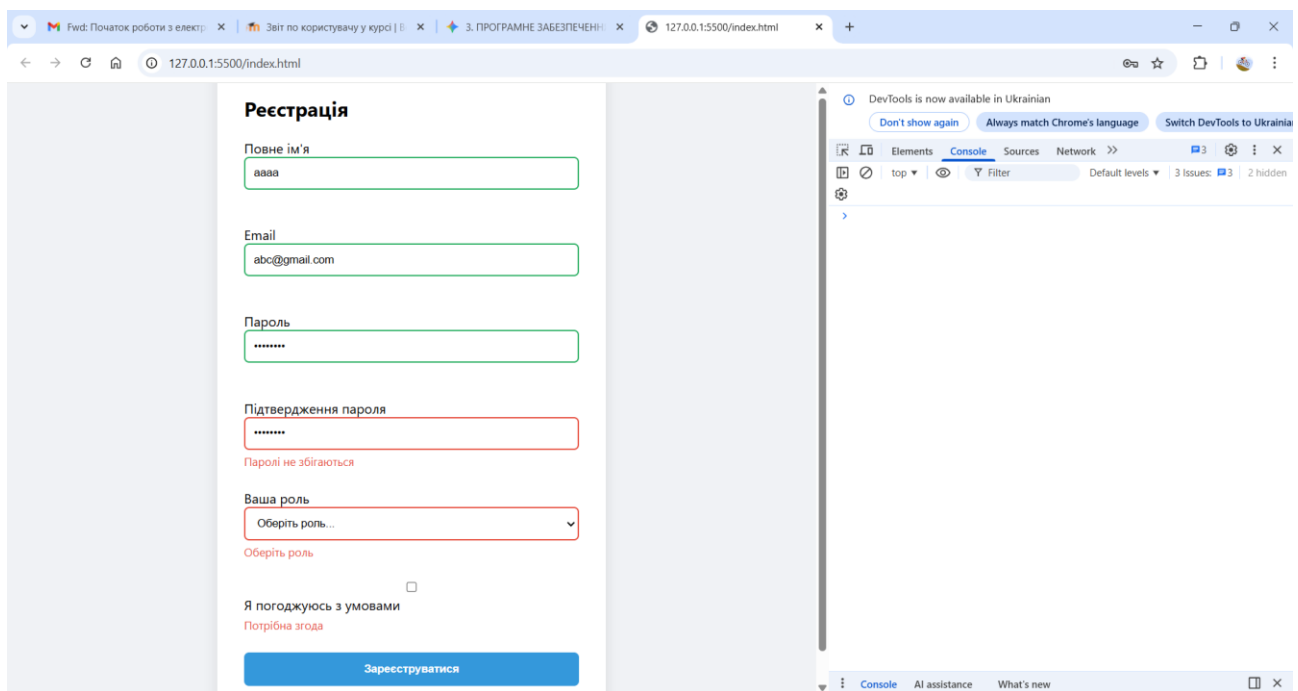


Рис.4 Тестування логічної перевірки збігу паролів.

На даному етапі тестування було перевірено здатність скрипта розрізняти валідні дані від логічних помилок користувача.

Опис результату:

Успішна валідація окремих полів: Поля «Повне ім'я», «Email» та «Пароль» заповнені згідно з правилами (довжина понад 2 символи, правильний формат

пошти та мінімум 8 символів у паролі). Вони отримали зелену рамку (клас `.valid`), що свідчить про проходження первинної перевірки.

Виявлення невідповідності паролів:

- Поле «Підтвердження пароля» підсвічене червоним кольором.
- Виведено повідомлення про помилку: *"Паролі не збігаються"*.
- Це підтверджує коректну роботу функції `validateConfirm()`, яка порівнює значення двох полів за допомогою строгого оператора рівності.

Комплексна перевірка форми: Попри те, що верхні поля заповнені вірно, загальний статус форми залишається невалідним, оскільки виявлено помилки у виборі ролі та відсутності згоди з умовами.

Стан інтерфейсу: Консоль розробника залишається чистою, оскільки форма не пройшла повну перевірку і дані не були серіалізовані в JSON.

Тест №3: Повна валідність форми перед надсиленням

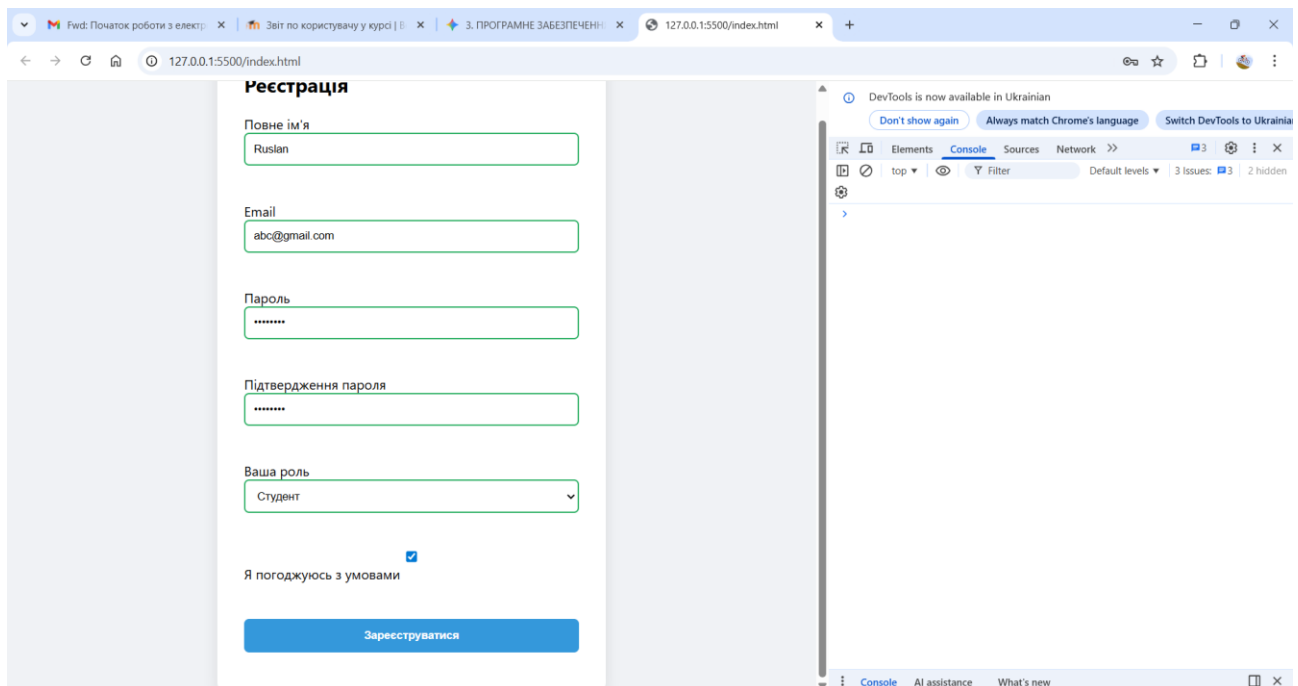


Рис.5 Стан повної валідності всіх полів форми перед натисканням кнопки відправки.

На скріншоті 5 зображено стан форми, коли всі дані введені коректно та відповідають встановленим правилам валідації.

Опис результату:

Успішна перевірка всіх полів: Усі поля (Ім'я "Ruslan", Email, Паролі, Роль "Студент" та чекбокс) підсвічені зеленим кольором. Це означає, що функція `validateAll()` повернула значення `true`.

Готовність до відправки: Кнопка «Зареєструватися» активна. Консоль розробника все ще порожня, оскільки запит розпочнеться лише після натискання кнопки.

Логічна цілісність: Поле підтвердження пароля тепер валідне, оскільки введені значення повністю збігаються.

Тест №4: Успішна реєстрація та серіалізація даних у JSON

На скріншоті продемонстровано фінальний результат роботи програми після успішної імітації мережевого запиту.

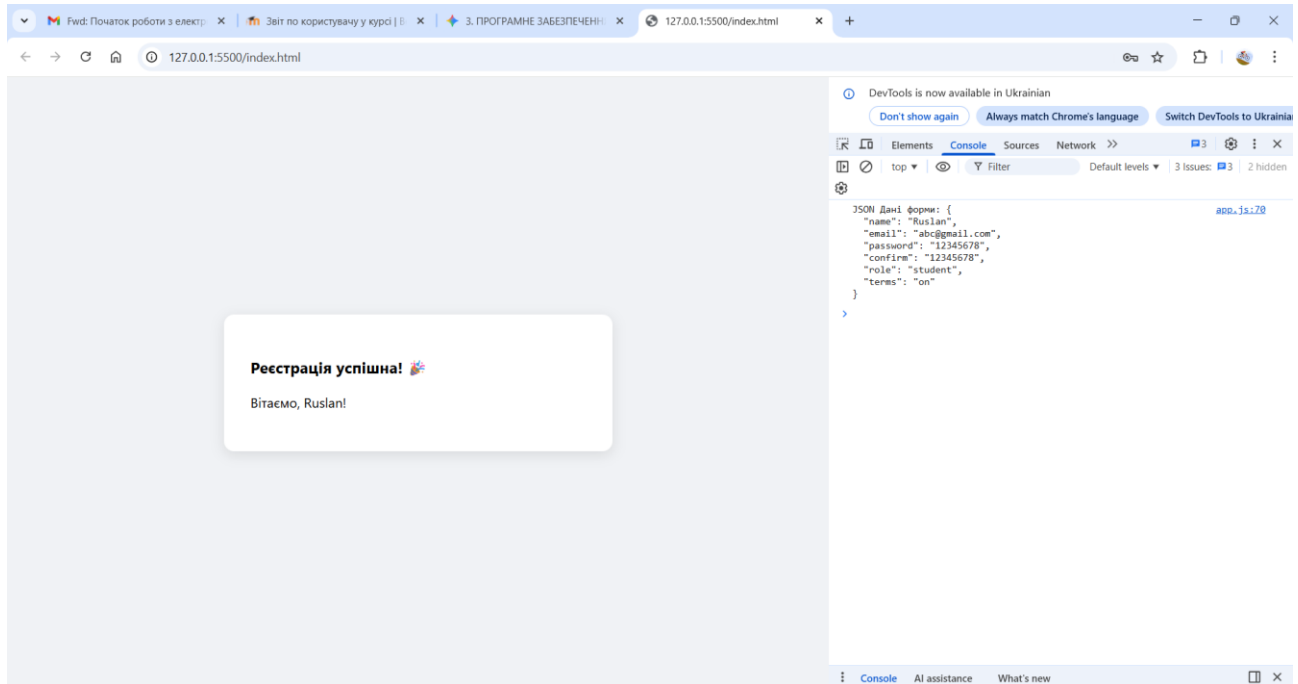


рис.6.фінальний результат: вивід JSON-об'єкта в консоль та відображення повідомлення про успішну реєстрацію.

Опис результату:

Приховування форми: Після натискання кнопки та завершення 2-секундної затримки (await setTimeout), форма була автоматично прихована.

Повідомлення про успіх: На екрані з'явилася картка з підтвердженням «Реєстрація успішна!» та персоналізованим привітанням «Вітаємо, Ruslan!».

Ім'я було динамічно взято з об'єкта даних.

Серіалізація в JSON: У консолі браузера (Chrome DevTools) відображено результат роботи конструктора FormData. Об'єкт містить усі ключі та значення: name: "Ruslan"

email: "abc@gmail.com"

password та confirm: "12345678"

role: "student"

terms: "on" (підтвердження відмітки чекбоксу).

Завершення життєвого циклу: Програма успішно виконала всі поставлені завдання: від валідації до формування кінцевого об'єкта даних для сервера.

ВИСНОВОК

В ході роботи було реалізовано:

- Комплексну валідацію: перевірку полів на довжину, формат Email (через RegExp) та логічну відповідність (збіг паролів).
- Динамічний UI: налаштовано миттєве відображення помилок та зміну стилів полів (класи .valid/.invalid) у реальному часі.
- Асинхронну обробку: реалізовано імітацію відправки даних на сервер із блокуванням кнопки submit для уникнення дублювання запитів.

- Роботу з даними: засвоєно збір даних через об'єкт `FormData` та їх серіалізацію у формат `JSON` для подальшої передачі на бекенд.

Отримані навички дозволяють створювати надійні та зручні інтерфейси для взаємодії з користувачем, мінімізуючи помилки при введенні даних.

Відповіді на контрольні питання:

1. Призначення `novalidate` Цей атрибут вимикає стандартну перевірку форми браузером (ті самі спливаючі підказки «Заповніть це поле»). Він потрібен, щоб браузер не заважав працювати вашому власному JavaScript-коду.

Що буде, якщо його не вказати? Браузер спробує перевірити форму першим. Якщо він знайде помилку (наприклад, пусте поле `required`), він покаже свою стандартну підказку і заблокує подію `submit`. В результаті ваш JavaScript-обробник навіть не запуститься, і ви не побачите своїх кастомних помилок чи серіалізації в `JSON`.

2. `textContent` сприймає будь-який вхідний рядок як звичайний текст. Навіть якщо в повідомленні буде HTML-код, браузер просто відобразить його буквами.

`innerHTML` намагається розпізнати рядок як HTML-теги. Якщо там буде скрипт, браузер його запустить.

Загроза безпеки: Використання `textContent` захищає від XSS-атак (Cross-Site Scripting). Якщо злоумисник спробує замість імені ввести шкідливий код (наприклад, `<script>alert('Hacked!')</script>`), то при використанні `innerHTML` цей скрипт виконається в браузері. `textContent` просто виведе цей текст на екран, не даючи йому шансу нашкодити.

3. `input` — найшвидша подія. Вона спрацьовує миттєво кожного разу, коли змінюється вміст поля (натиснули клавішу, видалили символ або вставили текст). Ідеально підходить для «живої» валідації.

`change` — спрацьовує з затримкою. Вона активується лише тоді, коли користувач змінив текст і втратив фокус із поля (наприклад, клікнув в іншому місці або натиснув `Enter`). Якщо текст у полі не змінювався, подія не виникне.

`blur` — спрацьовує в момент втрати фокусу (коли користувач іде з поля), незалежно від того, чи змінювалися в ньому дані. Це зручно для перевірки поля вже після того, як користувач закінчив введення.

6. Конструктор `FormData` взагалі ігнорує невідмічені чекбокси. Якщо галочка не стоїть, ключ цього поля просто не буде доданий до об'єкта `FormData`. Він поводить себе так само як стандартна відправка форми в HTML: передаються лише "успішні" (`checked`) елементи.

Оскільки ключа немає в `FormData`, метод `Object.fromEntries` не створить відповідну властивість у фінальному об'єкті. У результаті у JSON-об'єкті поле `terms` буде просто відсутнє. Щоб уникнути цього, у валідації зазвичай додають перевірку `if (!data.terms) data.terms = false;` або перевіряють стан чекбокса безпосередньо через DOM перед серіалізацією.

11. Три способи отримати значення поля

Через `id` або селектор: звернення напряму до елемента та його властивості `value` (наприклад, `document.getElementById('name').value`).

Через об'єкт події (`event.target`): отримання значення елемента, який викликав подію (наприклад, `e.target.value`).

Через конструктор `FormData`: збір значень усіх полів форми одночасно за їхнім атрибутом `name` (як ми робили у фінальній частині роботи).

В обробнику події `input` найчастіше використовується `event.target.value`.

Обґрунтування вибору:

- Універсальність: цей підхід дозволяє створювати єдину функцію-обробник для декількох полів одночасно, оскільки `event.target` завжди посилається на конкретний елемент, що ініціював подію.

- Продуктивність: відсутня потреба у повторному пошуку елемента в структурі документа за допомогою `getElementById` чи `querySelector`, що оптимізує роботу скрипта під час інтенсивного введення тексту.

Індивідуальне завдання (3 рівень).

Варіант 5.

Додавання кнопки скидання форми.

Опис реалізації:

1. У файл `index.html` додано кнопку `<button type="button" id="resetBtn">`.

```
<button type="button" id="resetBtn" class="reset-button">Скинути форму</button>
```

2. Для візуального оформлення кнопки скидання форми до файлу `style.css` було додано наступний блок стилів:

```
.reset-button {
  width: 100%;
  padding: 10px;
  margin-top: 10px;
  background-color: #6c757d;
  color: white;
```

```
border: none;
border-radius: 4px;
cursor: pointer;
}

.reset-button:hover {
  background-color: #5a6268;
}
```

3. У файлі `app.js` реалізовано обробник події `click`, який виконує метод `form.reset()`, а також за допомогою циклу `forEach` видаляє класи `.valid` та `.invalid` з усіх елементів введення. Додатково очищується вміст усіх контейнерів для помилок `.error-msg`.

```
const resetBtn = document.getElementById('resetBtn');

resetBtn.addEventListener('click', () => {
  form.reset();

  const inputs = form.querySelectorAll('input, select');
  inputs.forEach(input => {
    input.classList.remove('valid', 'invalid');
  });

  const errorMessages = form.querySelectorAll('.error-msg');
  errorMessages.forEach(msg => {
    msg.textContent = '';
  });

  console.log('Форму скинуто');
});
```

Перевірка функціоналу та скидання форми.

На скріншотах продемонстровано роботу кнопки «Скинути форму», яка повертає інтерфейс до початкового нейтрального стану.

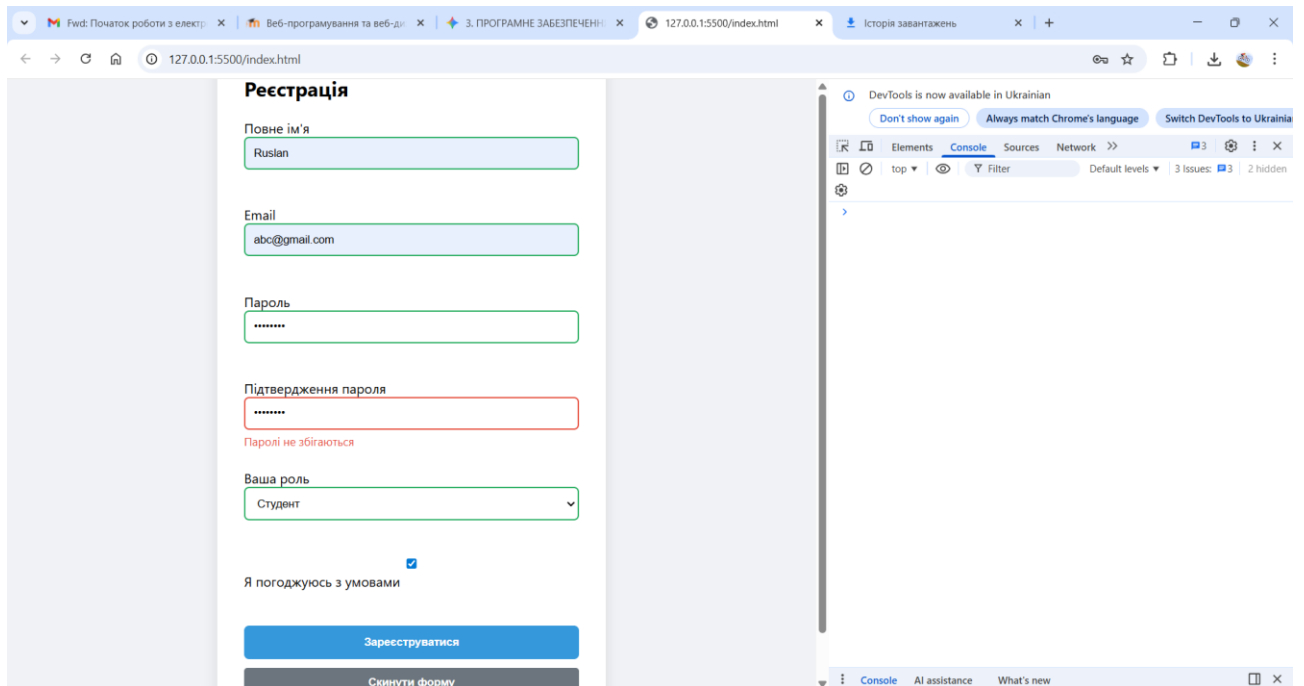


рис.7 Стан до скидання

форма частково заповнена даними (ім'я «Ruslan», email, пароль). Поля мають активну візуальну індикацію: зелені рамки для валідних даних та червону рамку з повідомленням "Паролі не збігаються" для невалідних. Це підтверджує, що перед тестуванням кнопки скидання форма перебувала в активному стані з наявними CSS-класами `.valid` та `.invalid`.

2) Після натискання на кнопку «Скинути форму» спрацював обробник події, який виконав метод `form.reset()`.

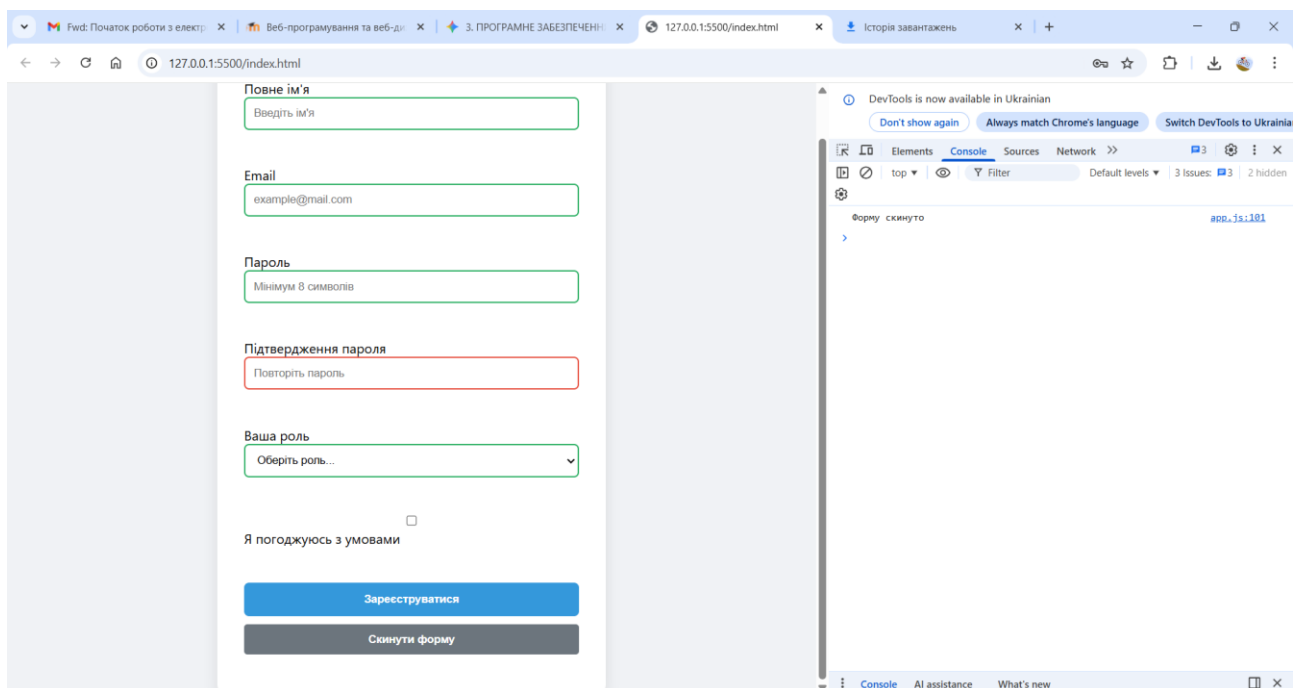


рис.8 Стан після скидання

Текстові повідомлення про помилки під полями повністю зникли.

Підтвердження в консолі: У Chrome DevTools з'явилося повідомлення «Форму скинуто», що свідчить про успішне виконання всього блоку коду в `app.js`

У результаті виконання завдання було розширено функціонал форми кнопкою скидання, що дозволяє коректно керувати станом DOM-елементів (видалення CSS-класів та тексту помилок). Отримані знання дозволяють створювати інтерактивні та зручні для користувача веб-інтерфейси з надійною перевіркою даних на стороні клієнта.